

SWEN 670 — Software Engineering Capstone

CareConnect Application

Software Requirements Specification (SRS)

Team B

Summer 2026 | University of Maryland Global Campus

Professor Mir Assadullah

Version 1.0

Version	Date	Description
1.0	5/13/2026	Initial draft
1.1	6/7/2026	

Table of Contents

- [1. Introduction](#)
 - [1.1 Purpose](#)
 - [1.2 Project Scope](#)
 - [1.3 Definitions and Abbreviations](#)
- [2. Overall Description](#)
 - [2.1 Product Perspective](#)
 - [2.2 User Classes and Characteristics](#)
 - [2.3 Operating Environment](#)
 - [Section 2.3.1 Minimum iOS Version](#)
 - [Section 2.3.2 Minimum Android API Level](#)
 - [2.3.3 CI/CD Pipeline](#)
- [3. Functional Requirements](#)
 - [3.1 Testing Requirements](#)
 - [3.1.1 Testing Baseline](#)
 - [3.1.2 Unit Testing Requirements](#)
 - [3.1.3 End-to-End Testing Requirements in](#)
- [4. Accessibility Requirements](#)
 - [4.1 WCAG 2.1 AA Requirements](#)
 - [4.2 Section 508 Requirements](#)
 - [4.3 VPAT Requirements](#)
- [5. Non-Functional Requirements](#)
 - [5.1 Performance](#)
 - [5.2 Security](#)
 - [5.3 Reliability and Availability](#)
 - [5.4 Maintainability](#)
 - [5.5 Portability](#)
- [6. User Requirements](#)
 - [6.1 Use Cases](#)
- [7.1 AWS Infrastructure](#)
- [7.2 Environment Configuration](#)
- [7.3 CI/CD Requirements](#)
- [8. Assumptions, Dependencies, and Constraints](#)
 - [8.1 Assumptions](#)
 - [8.2 Dependencies](#)
 - [8.3 Constraints](#)
- [9. Open Items](#)
- [Appendix A - Acronyms, Abbreviations, and Definitions](#)

[Appendix B - Acronyms, Abbreviations, and Definitions](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the CareConnect application as developed by Team B of SWEN 670, Summer 2026. This document focuses on the testing, accessibility, and quality requirements within Team B's scope. It is intended to guide development, testing, and validation activities across all milestones.

1.2 Project Scope

CareConnect is a full-stack healthcare application designed to streamline communication and coordination between caregivers and patients. It includes a Flutter/Dart frontend and a Spring Boot Java backend, supporting authentication, scheduling, messaging, and care coordination features.

Team B's scope within CareConnect covers:

- Comprehensive unit and end-to-end testing infrastructure
- Achieving and maintaining tiered code coverage — 100% on high-risk transactional logic (Shift Scheduling, EVV) and 90% on visual/placement components
- WCAG 2.1 AA and Section 508 accessibility compliance
- Production of a Voluntary Product Accessibility Template (VPAT)
- Validation of UI components delivered by SWEN 661

1.3 Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification
WCAG	Web Content Accessibility Guidelines — international standard for web/app accessibility
Section 508	U.S. federal law requiring electronic and information technology to be accessible to people with disabilities
VPAT	Voluntary Product Accessibility Template — document reporting a product's conformance with accessibility standards
E2E	End-to-end testing — testing complete user workflows from UI to backend
Flutter	UI toolkit by Google for building natively compiled applications for mobile, web, and desktop from a single codebase

CI/CD	Continuous Integration / Continuous Deployment — automated pipeline for building, testing, and deploying code
661	SWEN 661 — consulting class that delivers UI components to the 670 CareConnect project

2. Overall Description

2.1 Product Perspective

CareConnect is an extension and enhancement of the prior cohort's work (Spring 2026). The Summer 2026 cohort inherits the existing codebase and is responsible for adding new features, improving quality, and ensuring accessibility compliance. Team B's work is foundational. Team B's work supports the overall quality of the CareConnect application.

2.2 User Classes and Characteristics

The following table identifies the primary user classes of the CareConnect application, their responsibilities within the system, and their expected accessibility needs. Understanding these user classes informs both the functional requirements and the accessibility compliance standards Team B is responsible for validating.

User Class	Description	Accessibility Needs
Caregiver	Manages care schedule, adds patients, creates tasks, responds to alerts	Likely to use Desktop and Mobile
Care-recipient/Patient	Logs symptoms, views schedule, responds to reminders, activates alerts	Likely to need accessibility features (high contrast, screen reader, large tap targets)
Family Member	Supportive of Care-Recipient/Patient, views reports, monitors logs, receives alerts	Likely to be primarily Mobile focused
Administrator	Oversees system, manages logs, compliance, and permissions	Desktop focused

2.3 Operating Environment

CareConnect is designed to operate within a cloud-hosted AWS environment supporting mobile, web, and backend infrastructure components. The application supports development, testing, and deployment workflows across multiple environments including local development, staging, and production.

Supported operating environments include:

- iOS mobile devices
- Android mobile devices
- Modern desktop web browsers
- AWS-hosted backend infrastructure

Infrastructure dependencies include:

- AWS cloud services for hosting and database management
- GitHub for source control and CI/CD workflow integration
- GitHub Actions for automated testing and deployment workflows
- Environment variable configuration for secure API and authentication management

The application is expected to support development across Windows, macOS, and Linux-based environments.

Section 2.3.1 Minimum iOS Version

The application is expected to support modern iOS devices capable of running the current Flutter build. Based on the project setup, the minimum iOS version should be treated as **iOS 12.0 or higher**, pending final confirmation during Milestone 2 testing. This version provides compatibility with common Flutter dependencies and supports expected mobile functionality such as authentication, forms, scheduling, and messaging.

Section 2.3.2 Minimum Android API Level

The application is expected to support Android devices running **Android API Level 21 or higher**. This aligns with typical Flutter Android compatibility and allows support for the application's core features, including login, form submission, schedule viewing, and notifications. This value should be confirmed during Milestone 2 when the team validates the final Android build configuration.

2.3.3 CI/CD Pipeline

The CareConnect project will utilize GitHub Actions as the primary CI/CD automation platform. Automated workflows shall support continuous integration activities including build validation, automated testing, and pull request verification.

Planned CI/CD workflow responsibilities include:

- Automated unit test execution
- End-to-end test execution
- Code coverage report generation

- Pull request validation checks
- Branch protection enforcement for team-b-develop
- Build failure notification and logging

Additional workflow configurations will be documented as repository setup and testing infrastructure mature throughout future milestones.

3. Functional Requirements

3.1 Testing Requirements

This section defines Team B's testing requirements across all phases of the CareConnect project. Requirements are organized into three areas: the testing baseline established from the prior cohort's codebase, unit testing standards, and end-to-end testing standards. Each requirement includes a priority level, assigned owner, and verification method.

3.1.1 Testing Baseline

The following testing baseline was established through a comprehensive audit of the prior cohort's CareConnect codebase. It documents the current testing frameworks in use, code coverage percentages by layer, and identified gaps that Team B will address across Milestones 2 through 4.

Area	Current Status	Gap (to 95%)
Testing Framework - Backend	JUnit 5 / Spring Boot Test with JaCoCo coverage	
Testing Framework - Frontend	Flutter test (flutter test) with lcov coverage	
Backend Line Coverage %	90%	5%
Backend Branch Coverage %	77.6%	22.4%
Frontend Line Coverage %	56.8%	43.2%
Unit tests - Backend (code coverage %)	90%	5%
Unit tests - Frontend (code coverage %)	15%*	80%
Backend E2E test coverage %	90%	5%
Frontend E2E test coverage %	58.6%	43.2%
CI integration	No active CI integration	

**Much of Frontend testing is handled by widget tests. Including widget tests brings Frontend code coverage to 56.8%.*

Notable Coverage Gaps

Backend:

Area	Line %	Notes
AI / Bedrock orchestration com.careconnect.ai.bedrock	25%	The Bedrock client wrapper contains prompt-building logic and streaming response handling. Only 6 of 24 lines covered — primarily because the service is mocked at the SDK boundary in all tests; the internal orchestration logic inside the service itself is not directly exercised.
Notification DTOs com.careconnect.notifications.dto	41.9%	Notification data-transfer objects (appointment reminders, SMS payloads). Low coverage because the two failing notification tests (SesServiceTest, SnsServiceTest) test the correct message content but their assertions are currently failing due to template drift.
AI service layer com.careconnect.ai	52.4%	High-level AI routing and prompt dispatch logic. The low branch coverage (1/10) indicates that most conditional paths (model selection, fallback logic, error recovery) are untested.
Utilities com.careconnect.util	78.5%	Utility methods with many edge-case branches (null checks, format conversions). The 62% branch coverage leaves meaningful conditional paths untested.

Frontend:

Feature	Line %	Impact
Shift Scheduling	10.1%	Only 79 of 782 lines covered. The scheduling UI (calendar views, shift assignment, conflict detection) has almost no test coverage. This is the lowest-covered feature in the entire project.
EVV	29.5%	1,469 of 4,982 lines. The EVV feature is the largest feature by line count. The 29.5% figure reflects coverage from widget tests that indirectly render EVV pages; the business logic (visit start/end, GPS capture, HHAExchange submission, offline sync) is largely uncovered.
File Handling	31.5%	The factory and platform-specific file handler implementations. The underlying download/upload logic paths are untested.
Other / Uncategorized	36.9%	5,310 lines in files outside the standard feature directory structure (top-level pages, in_home feature, legacy screens). These files are not systematically tested.
Payments	48.4%	Checkout and subscription management pages. The Stripe SDK calls are not mocked in widget tests, leaving the payment flow branches uncovered.
Social	50.1%	In-app chat UI and social DTO parsing. The WebSocket-backed messaging logic is not exercised by unit tests — it requires a live server or a WebSocket mock.

Overall Test results:

Layer	Total	Pass	Skip	Fail
Backend	5,688	5,686	0	2
Frontend	10,339	9,989	36	317
Combined	16,027	15,674	36	319

3.1.2 Unit Testing Requirements

The following requirements define the unit testing standards that apply to all CareConnect modules. These requirements establish the minimum coverage thresholds, process expectations, and verification methods that Team B will enforce throughout the project lifecycle.

ID	Requirement	Priority	Owner
TR-001	The application shall achieve a minimum of 95% code coverage across all modules.	High	Brice / Prashanth
TR-002	Unit tests shall be written or updated for all new and modified components before merging to team-b-develop	High	All members
TR-003	Code coverage reports shall be generated automatically as part of the CI/CD pipeline	High	Brice
TR-004	All unit tests shall pass before any merge to team-b-develop	High	All members
TR-005	Unit test coverage shall be measured by line and by feature for all deliverable source code. Each functional requirement shall map to at least one test case via a traceability matrix. Verification: Coverage report (showing per-file line coverage and any exclusions) and a test result report (showing pass/fail counts and failure details) shall be generated and included with each milestone submission.	Medium	Prashanth

Backend Unit Testing Failures:

Test Class	Test Method	Failure
com.careconnect.notifications.SesServiceTest	sendAppointmentReminder_success_buildsCorrectSubjectAndContent (line 300)	Expected subject: "Appointment Reminder - Checkup" Actual subject: "Appointment Reminder" Email subject template no longer appends the appointment type.
com.careconnect.notifications.SnsServiceTest	sendAppointmentReminderSms_success_messageContainsDetails (line 348)	Expected: true Actual: false SMS body content assertion — message text no longer matches the expected substring.

FrontEnd Unit Testing Failures:

Count	Test File	Root Cause Category
71	features/auth/sign_up_screen_test.dart	DotEnv not initialized
45	features/health/virtual_check_in/virtual_check_in_config_sheet_test.dart	ListTile inside DecoratedBox
35	features/evv/evv_record_review_test.dart	Async timing / empty list
21	services/evv_service_test.dart	Async timing / mock data
19	features/dashboard/patient_dashboard_page_test.dart	Async timing / empty list

16	features/payments/subscription_management_page_test.dart	RangeError — index out of range
14	features/health/caregiver-patient-list/patient_details_page_test.dart	Async timing / empty list
14	features/evv/schedule_page_test.dart	Async timing / mock data
14	features/ai/presentation/pages/voice_command_ai_test.dart	Route / navigation mismatch
12	features/payments/stripe_checkout_page_test.dart	Async timing / mock data
12	services/local_db/app_database_stub_test.dart	Stub logic incomplete
10	video_call_e2e_coverage_proxy_test.dart	No backend available (expected)
5	features/dashboard/caregiver_dashboard/upcoming_checkins_widget_test.dart	Async timing
~20	~15 other files (≤3 each)	Mixed

3.1.3 End-to-End Testing Requirements

The following requirements define Team B's end-to-end testing standards for the CareConnect application. E2E tests validate complete user workflows from entry point to expected result, ensuring that integrated features function correctly across the full application stack.

ID	Requirement	Priority	Owner
ET-001	E2E tests shall cover all critical CareConnect user workflows	High	Brice
ET-002	Team B's tested workflows function correctly within the application, covering the full execution path from entry point to expected result. Verification: Team B creates comprehensive E2E test suits which will be run with each team-b-develop before merging on each milestone.	High	Brice / Prashanth
ET-003	Regression tests shall be run on team-b-develop after each Monday merge to verify no regressions in Team B's tested workflows	High	Team B
ET-004	E2E test workflows shall be documented, covering the full execution path from entry point to expected result. Verification: Team B will perform E2E test sessions and document actual results against expected responses with each milestone submission.	Medium	Prashanth
ET-005	Root cause analysis on failing E2E test results shall be performed to identify the best avenue for remediation. Verification: Root cause analysis report and possible remediation plan shall be included with each milestone submission.	Medium	Prashanth

End 2 End Test Results:

Test suite	Tests	Passed	Failed	Blocked
Backend — CallFlowIntegrationTest	19	19	0	0
Frontend — chat_messaging_e2e_test.dart	1	1	0	0

Frontend — video_call_e2e_test.dart	10	—	—	10
-------------------------------------	----	---	---	----

Blocked Frontend E2E Tests:

Group	TDD ID	Test name	Backend dependency
App launch	CALL-001	app launches and displays an initial screen	OFFLINE
App launch	CALL-001	MaterialApp is present after launch	OFFLINE
Login flow	CALL-001	login page renders email and password fields	OFFLINE
Login flow	CALL-001	login helper navigates past login screen when backend is available	REQUIRES backend
Call screen	CHIME-004	call screen shows loading indicator then transitions to call or error UI	REQUIRES backend
Sentiment panel	SENT-002	sentiment panel is visible after caregiver joins an active call	OFFLINE (structural)
Sentiment panel	SENT-002	sentiment panel is NOT visible when user is a patient	OFFLINE (structural)
End call	CHIME-006	end call button is present and tappable in an active call	OFFLINE (structural)
End call	CHIME-006	tapping end call does not throw an unhandled exception	OFFLINE
Chime embed	CHIME-004	Chime embed area is present after successful call join	OFFLINE (structural)

Blocker	Target	Root cause
pdfx CMake failure	Windows desktop	pdfx plugin downloads pdfium at CMake configure time. The pdfium sub-project declares cmake_minimum_required below 3.5, which CMake ≥ 4.0 rejects.

CupertinoPageTransitionsBuilder web compile error	Chrome (web)	main.dart uses CupertinoPageTransitionsBuilder without importing package:flutter/cupertino.dart. The Dart VM resolves it via material.dart's indirect chain; the Chrome DDC compiler requires an explicit import. Additionally the const qualifier on PageTransitionsTheme was incompatible with Flutter 3.44's constructor signature.
DWDS AppConnectionException	Chrome (web)	After one successful flutter drive session the Chrome remote-debugging port was not released. The subsequent flutter drive session could not establish a DWDS debug connection and hung.

Note:

During Frontend testing, 418 test files originally did not compile. The file “lib/features/evv/presentation/pages/evv_hhaexchange_submit_page.dart” imported “file_handler_web.dart” which in turn imported “dart:js_interop”. “dart:js_interop” is a Dart SDK library only available for web compilers. The Dart VM used for flutter tests was unable to handle importing “dart:js_interop” which led to compile failures. This was fixed by editing the “evv_hhaexchange_submit_page.dart” file at two places.

Line 7 – Import:

```
// Before (broken — leaks dart:js_interop into Dart VM compilation)
```

```
import '../utils/file_handler_web.dart';
```

```
// After (fixed — VM-safe)
```

```
import '../factories/file_handler_factory.dart';
```

Line 149 – Instantiation:

```
// Before
```

```
final fileHandler = WebFileHandler();
```

```
// After
```

```
final fileHandler = FileHandlerFactory.create();
```

“FileHandlerFactory.create()” is already defined in “lib/factories/file_handler_factory.dart” and uses Dart's built-in conditional import mechanism. This allowed those 418 test files to compile and provide a full picture of test coverage. The changes to “lib/features/evv/presentation/pages/evv_hhaexchange_submit_page.dart” were reverted after testing was complete.

AI Use Disclaimer:

This test report was created with the help of Claude Code on Sonnet 4.6, which automated testing, found the “`evv_hhaexchange_submit_page.dart`” fix, and compiled test summaries.

Team B Quality Assurance and Accessibility Validation Workflow

The following workflow illustrates Team B's responsibilities for testing, accessibility validation, and compliance reporting throughout the project lifecycle.

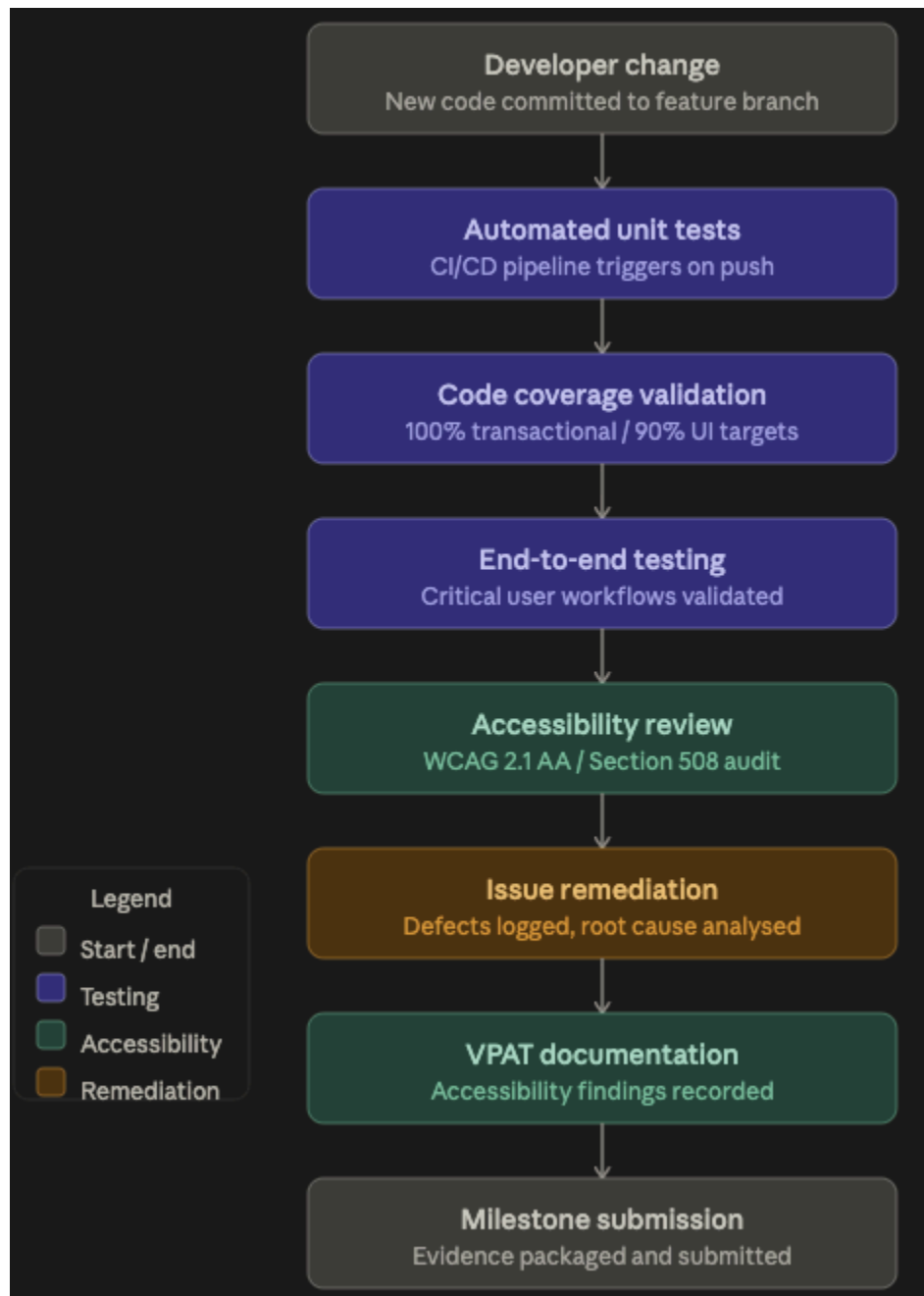


Figure 3.1. Team B testing, accessibility, and compliance workflow.

4. Accessibility Requirements

4.1 WCAG 2.1 AA Requirements

CareConnect shall conform to WCAG 2.1 Level AA. The following requirements represent the minimum accessibility standards the application must meet. Each criterion is mapped to a specific Team B audit or testing action that will be performed and documented across Milestones 2 through 4.

ID	WCAG Criterion	Requirement	Team B Action
AC-001	1.1.1	All non-text content shall have a text alternative	Audit and document findings
AC-002	1.3.1	Information and structure shall be programmatically determinable	Audit and document findings
AC-003	1.4.3	Text shall have a contrast ratio of at least 4.5:1	Audit using color contrast tools
AC-004	1.4.4	Text shall be resizable up to 200% without loss of content or functionality	Test on mobile and desktop
AC-005	2.1.1	All functionality shall be operable via keyboard	Keyboard navigation audit
AC-006	2.4.3	Focus order shall preserve meaning and operability	Focus indicator audit
AC-007	2.4.6	Headings and labels shall be descriptive	Screen reader audit
AC-008	3.1.1	Language of page shall be programmatically determinable	Audit and document findings
AC-009	4.1.2	All UI components shall have name, role, and value programmatically determinable	Screen reader and ARIA audit

4.2 Section 508 Requirements

CareConnect shall comply with Section 508 of the Rehabilitation Act, which requires that electronic and information technology be accessible to people with disabilities. The following criteria represent the key functional accessibility standards the application must meet in addition to WCAG 2.1 AA conformance.

4.3 VPAT Requirements

The following requirements govern Team B's production of the Voluntary Product Accessibility Template (VPAT). The VPAT is a formal document that reports the CareConnect application's conformance with applicable accessibility standards, including WCAG 2.1 AA and Section 508. It is a required deliverable for Milestone 4.

ID	Requirement	Priority	Owner
VP-001	Team B shall produce a VPAT document using the official ITI VPAT 2.4 template	High	Alex / Team B
VP-002	The VPAT shall document conformance level (Supports / Partial / Does Not Support) for each applicable WCAG criterion	High	Team B
VP-003	The VPAT shall include Section 508 functional criteria	High	Team B
VP-004	Expected VPAT format and completion level shall be confirmed with Professor Assadullah by Milestone 2	High	Alex

5. Non-Functional Requirements

5.1 Performance

The following performance requirements define the expected responsiveness and load characteristics of the CareConnect application under normal operating conditions. These thresholds apply to both the Flutter frontend and the Spring Boot backend and will be validated during integration and testing phases.

- The application shall load within 2.5 seconds on a standard mobile device.
- API response times shall not exceed 500 milliseconds for standard operations (Authentication, schedule retrieval, messaging, symptom logs) under normal load.
- Subsequent screen transitions shall complete with [1] second under normal network conditions.
- Background synchronization tasks shall complete within 3 seconds, including:
 - Pulling updated schedules
 - Retrieving new messages
 - Syncing symptoms.meal logs
- The application shall maintain smooth UI responsiveness, with no more than 2 UI updates per second.
- Voice interaction responses shall be returned within 1 second, supporting accessibility and STML-friendly workflows.

5.2 Security

The following security requirements define the standards the CareConnect application must meet to protect user data, ensure secure authentication, and comply with applicable healthcare privacy regulations including HIPAA. These requirements apply across both the frontend and backend layers of the application.

- All data in transit shall be encrypted using TLS 1.2 or higher
- Health data shall be handled in compliance with applicable privacy regulations
- All sensitive data at rest (patient records, schedules, messages, symptom logs) shall be encrypted using AWS-256 or an equivalent industry standard.
- User authentication shall use JWT tokens with expiration, including:
 - Short lived access tokens
 - Rotating refresh tokens
 - Server side invalidation on logout or credential reset
- The system shall enforce secure password policies, including:
 - Minimum 8 characters
 - At least one uppercase, lowercase, number, and symbol.
 - Lockout after 5 failed attempts.
- Session timeout shall occur after 15 minutes of inactivity, requiring re-authentication. This aligns with HIPAA expectations for unattended devices.

- The application shall comply with applicable healthcare privacy regulations, including HIPPA principles for:
 - Minimum necessary access
 - Audit logging
 - Secure transmission of PHI (Personal Health Information)
 - Role based access control
- The backend shall log all authentication events, including:
 - Login attempts
 - Token refreshes
 - Permission Changes
 - Failed Authentication attempts
- The application shall not store PHI (Personal Health Information) in local device logs, error messages, or crash reports.
- The application shall support OS-level accessibility and security features, including:
 - Biometric Authentication (Face/Touch ID)
 - Secure Keychain/keystore storage

5.3 Reliability and Availability

The following reliability and availability requirements define the expected uptime, fault tolerance, and recovery behavior of the CareConnect application. These requirements ensure the application remains functional and data is preserved under adverse conditions including network interruptions, backend restarts, and dependent service failures.

- The application shall target 99% uptime during the semester, excluding scheduled maintenance.
- The system shall automatically recover from backend restarts (i.e. redeployments) without requiring user intervention.
- The application shall handle network interruptions gracefully including:
 - Preserving unsent messages locally
 - Retrying failed API calls
 - Displaying clear inline error messages (no pop-ups)
- The system shall not lose user data during temporary connectivity loss, using local caching for:
 - Symptom entries
 - Meal logs
 - Task completion events
- The backend shall perform automated daily backups of the MySQL database, with:
 - 7 day retention

- Ability to restore within 24 hours
- The system shall degrade gracefully when dependent services (AI, notifications, analytics) are unavailable, ensuring core functionality (scheduling, messaging, logs) remains operational.
- CI/CD pipeline shall prevent deployment of builds with failing tests, ensuring reliability of integrated features.

5.4 Maintainability

The following maintainability requirements establish the standards Team B will follow to ensure the CareConnect codebase remains readable, testable, and well-documented throughout the project lifecycle. These standards apply to all code and documentation produced by Team B.

- All code shall follow the project's TESTING_NORMS.md conventions
- README and testing documentation shall be updated with each significant change
- Code coverage reports shall be generated and stored with each milestone submission

5.5 Portability

The following portability requirements define the platforms, orientations, and viewport sizes the CareConnect application must support. These requirements ensure the application is accessible and functional across the full range of devices used by caregivers, patients, and administrators.

- The Flutter frontend shall support iOS, Android, and web platforms
- The application shall function correctly in landscape and portrait orientations
- The application shall be tested on mobile, tablet, and desktop viewports

6. User Requirements

6.1 Use Cases

The following use cases describe the primary interactions between system actors and the CareConnect application. Each use case identifies the initiating actor, the trigger condition, and the expected outcome. Sequence diagrams and use case diagrams for each use case are provided in the sections that follow.

ID	Use Case	Actor	Description
UC-001	User Login	[All users]	<i>This use case begins when a user opens the application and needs to access protected system features. The user enters valid login credentials, and the system authenticates the account using the application's authentication process. If successful, the user is granted access to the appropriate dashboard based on their role.</i>
UC-002	Complete W-4 Form	[New Employee]	<i>This use case begins when a new employee is required to complete tax withholding information during onboarding. The employee enters the required W-4 information into the digitized form and submits it through the application. The expected outcome is that the completed form is stored and made available for employer review or processing.</i>
UC-003	Complete I-9 Form	[New Employee / Employer]	<i>This use case begins when a new employee must verify employment eligibility. The employee provides the required identity and employment authorization information, while the employer completes the</i>

			<i>required verification portion. The expected outcome is a completed I-9 workflow that supports onboarding compliance.</i>
<i>UC-00 4</i>	<i>Submit Direct Deposit</i>	<i>[Employee]</i>	<i>This use case begins when an employee wants to provide banking information for payroll. The employee enters their account and routing information into the direct deposit form and submits it securely. The expected outcome is that payroll has the information needed to process electronic payments.</i>
<i>UC-00 5</i>	<i>View Care Schedule</i>	<i>[Caregiver / Patient]</i>	<i>This use case begins when a caregiver, patient, or authorized user needs to review upcoming care visits. The user opens the scheduling feature and views scheduled appointments, visit times, and related details. The expected outcome is that the user understands upcoming care responsibilities and timing.</i>
<i>UC-00 6</i>	<i>Secure Messaging</i>	<i>Caregiver / Administrator / Employee</i>	<i>This use case begins when a user needs to communicate with another authorized user, such as a caregiver, patient, or administrator. The user opens the messaging feature, writes a message, and sends it through the system. The expected outcome is secure communication within the application.</i>
<i>UC-00 7</i>	<i>Manage Care Coordination</i>	<i>Caregiver / Administrator</i>	<i>This use case begins when a caregiver or administrator</i>

			<i>needs to update patient care activities and coordination tasks. The user accesses the care coordination section, updates visit notes, tasks, or patient care information, and saves the changes. The expected outcome is that patient care information remains accurate and accessible to authorized users.</i>
<i>UC-008</i>	<i>User Account Management</i>	<i>Administrator</i>	<i>This use case begins when an administrator needs to manage user access and permissions within the application. The administrator creates, updates, disables, or removes user accounts and assigns appropriate permissions. The expected outcome is that users only have access to features appropriate for their role.</i>
<i>UC-009</i>	<i>Upload Employment Documents</i>	<i>New Employee</i>	<i>This use case begins when a new employee is required to submit onboarding documentation. The employee uploads identification forms, employment paperwork, or supporting documents through the application. The expected outcome is that documents are securely stored and available for employer review.</i>
<i>UC-0010</i>	<i>Receive Notifications</i>	<i>All Users</i>	<i>This use case begins when the system generates updates related to schedules, messages, forms, or care activities. The user receives notifications through the application and reviews the provided information. The expected outcome is that users stay</i>

			<i>informed about important updates and required actions.</i>
--	--	--	---

7. Deployment and Infrastructure Requirements

7.1 AWS Infrastructure

The following section describes the AWS infrastructure supporting the CareConnect application. Infrastructure details were identified through an audit of the repository's cloudformation-fargate directory and will be confirmed and expanded in Milestone 2 as the team gains deeper access to the deployment environment.

The repository includes a cloudformation-fargate folder, which indicates that the application infrastructure is designed around AWS container-based deployment. Confirmed AWS services include **AWS CloudFormation** for infrastructure-as-code deployment and **AWS Fargate** for running containerized application services without directly managing servers.

Based on the Fargate deployment structure, the system likely also uses supporting AWS services such as **Amazon ECS** for container orchestration, **VPC/networking resources** for secure communication, and potentially load balancing/security group resources depending on the CloudFormation template configuration. These supporting services should be treated as inferred until fully confirmed in Milestone 2.

This infrastructure approach supports scalable deployment, simplified server management, and repeatable environment creation through CloudFormation templates.

7.2 Environment Configuration

The following environment variables are required to configure and run the CareConnect application across local development, staging, and production environments. All sensitive values must be stored securely and must not be committed to the repository. Team members should obtain these values from the team lead prior to local setup.

Variable	Purpose	Required For
OPENAI_API_KEY	AI/LLM integration	Backend
JWT_SECRET	Authentication token signing	Backend
CC_BACKEND_TOKEN	Backend API authentication	Frontend
CC_BASE_URL_*	Backend URL configuration per platform	Frontend
CC_SENTIMENT_MODE	Sentiment analysis mode (balanced/realtime/adaptive)	Frontend

7.3 CI/CD Requirements

The following requirements define the continuous integration and deployment standards for Team B's work on the CareConnect project. These requirements ensure that code quality, test coverage, and deployment readiness are validated automatically at each stage of the development workflow.

- All pushes to feature branches shall trigger automated test runs
- Code coverage reports shall be generated on each test run
- The team-b-develop branch shall only accept merges with passing tests and no conflicts

The CI/CD pipeline shall support automated quality assurance and deployment validation workflows for Team B.

Additional CI/CD requirements include:

- Automated accessibility testing integration where feasible
- Prevention of merges when required tests fail
- Automated code coverage reporting on pull requests
- Logging and reporting of failed workflow executions
- Validation of deployment configuration prior to release
- Support for future staging and production deployment environments

Future milestones may expand CI/CD workflows to include automated accessibility auditing and regression testing pipelines.

8. Assumptions, Dependencies, and Constraints

8.1 Assumptions

The following assumptions were made during the development of this SRS. These assumptions represent conditions that are expected to hold true throughout the project. If any assumption proves to be incorrect, the affected requirements and plans will be revisited and updated accordingly.

- The prior cohort's codebase is stable and can be compiled and run by all team members
- SWEN 661 will deliver UI components on a schedule compatible with Team B's testing timeline
- API keys and environment credentials will be provided to all team members before Milestone 1 deadline
- All team members will have their development environments set up by May 25, 2026

8.2 Dependencies

The following table identifies the external dependencies that Team B relies on to execute its scope of work. Each dependency includes its source and the potential impact if it becomes unavailable. Team B will monitor these dependencies throughout the project and escalate blockers as needed.

Dependency	Source	Impact if Unavailable
SWEN 661 UI components	SWEN 661 class	Team B cannot validate/test 661 deliverables
AWS environment access	Joriel / Prior cohort	Backend integration testing blocked
API credentials (.env.local)	Professor / Prior cohort	App cannot run locally
GitHub repo access	Joriel Rivas (class PM)	Code commits and reviews blocked
VPAT format confirmation	Professor Assadullah / Roy	VPAT production delayed

8.3 Constraints

The following constraints define the boundaries within which Team B must operate throughout the project. These are non-negotiable conditions imposed by the course structure, institutional policy, or the class-wide development process established by the class PM.

- All work must be completed within the semester timeline (May 13 — August 4, 2026)

- No direct commits to main or develop branches — all work via feature branches and PRs
- All documents submitted for milestones must use Word Track Changes for updates
- Peer evaluations must be submitted every week via the provided Google Form

9. Open Items

The following table documents questions and action items that required resolution prior to or during Milestone 1. Items marked Resolved have been confirmed and incorporated into this SRS. Any remaining open items will be addressed no later than Milestone 2.

#	Open Item	Owner	Status
1	Confirm current code coverage % from repo audit	Prashanth	Resolved
2	Identify testing framework already in place	Prashanth	Resolved
3	Confirm VPAT format and expected completion level	Alex	<i>Resolved</i>
4	Confirm hourly rates for cost estimation	Alex	Resolved
5	Identify highest-priority E2E test workflows	Brice	<i>Resolved</i>
6	Confirm accessibility tools (axe, WAVE, etc.) to use	Alex / 661	Resolved
7	Obtain API keys for all team members' .env.local files	Alex	<i>[Open] - Target M2</i>
8	Document AWS architecture from cloudformation-fargate folder	Lekan	Resolved

Appendix A - Acronyms, Abbreviations, and Definitions

The following table defines all acronyms, abbreviations, and specialized terms used throughout this Software Requirements Specification. Terms are listed alphabetically for ease of reference.

Term	Definition
SRS	Software Requirements Specification. A document that defines the functional and non-functional requirements of a software system.
RTM	Requirements Traceability Matrix. A document used to map requirements to their source, implementation, and verification method.
API	Application Programming Interface. A defined mechanism that allows software systems to communicate with one another.
AWS	Amazon Web Services. Cloud infrastructure services used for hosting and deployment.
EVM	Earned Value Management. A project management technique used to measure project performance and progress.
UI	User Interface. The screens, controls, and visual elements users interact with.
UX	User Experience. The overall experience and usability of the application from a user's perspective.
WCAG	Web Content Accessibility Guidelines. International standards for digital accessibility.
Section 508	U.S. federal accessibility requirements for information technology.
VPAT	Voluntary Product Accessibility Template. A document used to report accessibility conformance.
CI/CD	Continuous Integration and Continuous Deployment. Automated processes for building, testing, and deploying software.
E2E	End-to-End Testing. Testing that validates complete user workflows through the system.
RBAC	Role-Based Access Control. A security model that restricts access based on user roles.

Code Coverage	A testing metric that measures how much application code is executed during automated tests.
Regression Testing	Testing performed after changes to ensure existing functionality continues to work correctly.
Frontend	The user-facing portion of the application developed using Flutter.
Backend	The server-side portion of the application developed using Spring Boot.
Stakeholder	Any individual or group with an interest in the project's success or outcome.
Milestone	A major project checkpoint used to measure progress and deliverables.
Sprint	A fixed period of development work focused on achieving specific objectives.

Appendix B - Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) maps each requirement defined in this SRS to its source stakeholder, related feature area, priority level, and verification method. This matrix ensures that all requirements are traceable from origin through implementation and testing, and serves as a reference for validating completeness at each milestone submission.

Req ID	Requirement Description	Source / Stakeholder	Related Feature Area	Priority	Verification Method
TR-001	The application shall achieve tiered code coverage targets across all CareConnect modules: a minimum of 100% coverage on high-risk transactional logic modules (Shift Scheduling, Electronic Visit Verification), and a minimum of 90% coverage on visual/UI placement	Team B Scope / Client Quality Standard	Unit Testing	High	Coverage report per milestone submission

	components. All other modules shall target a minimum of 95% line and branch coverage.				
TR-002	Unit tests shall be written or updated for all new and modified components before merging to team-b-develop	Team B Process	CI/CD / Branch Protection	High	Pull request review / CI gate
TR-003	Code coverage reports shall be generated automatically as part of the CI/CD pipeline	Team B Scope	CI/CD Automation	High	GitHub Actions workflow output
TR-004	All unit tests shall pass before any merge to team-b-develop	Team B Process	Branch Protection	High	CI pipeline pass/fail gate
TR-005	Unit test coverage shall be measured by line and by feature; each requirement shall map to at least one test case	Team B Scope	Test Traceability	Medium	Coverage report + traceability matrix
ET-001	E2E tests shall cover all critical CareConnect user workflows	Team B Scope	End-to-End Testing	High	E2E test execution report per milestone
ET-002	Team B's tested workflows shall function correctly end-to-end within the application	Team B Scope	End-to-End Testing	High	E2E test results documentation

ET-003	Regression tests shall be run on team-b-develop after each Monday merge	Team B Process	Regression Testing	High	Regression test results per integration cycle
ET-004	E2E test workflows and expected results shall be documented	Team B Scope	Test Documentation	Medium	E2E documentation submitted per milestone
ET-005	Root cause analysis on failing E2E tests shall be performed and documented	Team B Scope	Defect Management	Medium	Root cause analysis report per milestone
AC-001	All non-text content shall have a text alternative (WCAG 1.1.1)	WCAG 2.1 AA / Section 508	Accessibility	High	Accessibility audit / screen reader test
AC-002	Information and structure shall be programmatically determinable (WCAG 1.3.1)	WCAG 2.1 AA	Accessibility	High	Accessibility audit / automated scan
AC-003	Text shall have a contrast ratio of at least 4.5:1 (WCAG 1.4.3)	WCAG 2.1 AA	Accessibility	High	Color contrast tool audit
AC-004	Text shall be resizable up to 200% without loss of content (WCAG 1.4.4)	WCAG 2.1 AA	Accessibility	High	Manual test on mobile and desktop
AC-005	All functionality shall be operable via keyboard (WCAG 2.1.1)	WCAG 2.1 AA / Section 508	Accessibility	High	Keyboard navigation audit

AC-006	Focus order shall preserve meaning and operability (WCAG 2.4.3)	WCAG 2.1 AA	Accessibility	Medium	Focus indicator audit
AC-007	Headings and labels shall be descriptive (WCAG 2.4.6)	WCAG 2.1 AA	Accessibility	Medium	Screen reader audit
AC-008	Language of page shall be programmatically determinable (WCAG 3.1.1)	WCAG 2.1 AA	Accessibility	Medium	Automated scan / audit
AC-009	All UI components shall have name, role, and value programmatically determinable (WCAG 4.1.2)	WCAG 2.1 AA / Section 508	Accessibility	High	Screen reader and ARIA audit
VP-001	Team B shall produce a VPAT using the official ITI VPAT 2.4 template	Client / Roy Gordon	VPAT Compliance	High	VPAT document submitted at Milestone 4
VP-002	VPAT shall document conformance level for each applicable WCAG criterion	Client / Roy Gordon	VPAT Compliance	High	VPAT review against WCAG criteria
VP-003	VPAT shall include Section 508 functional criteria	Client / Roy Gordon	VPAT Compliance	High	VPAT review against Section 508 criteria
VP-004	VPAT format and completion level shall be confirmed with Professor Assadullah by Milestone 2	Team B / Professor Assadullah	VPAT Planning	High	Confirmation received before M2 submission

NFR-001	Application shall load within 2.5 seconds on a standard mobile device	Performance Constraint	Performance	High	Load time test on mobile device
NFR-002	API response times shall not exceed 500ms for standard operations	Performance Constraint	Performance	High	API response time test
NFR-003	All data in transit shall be encrypted using TLS 1.2 or higher	Security / HIPAA	Security	High	Security inspection / TLS verification
NFR-004	Health data shall be handled in compliance with applicable privacy regulations including HIPAA	Regulatory / Client	Security / Privacy	High	Compliance inspection
NFR-005	User authentication shall use JWT tokens with expiration	Security Constraint	Authentication	High	Code review / security test
NFR-006	Application shall target 99% uptime during the semester	Reliability Constraint	Availability	Medium	Uptime monitoring
NFR-007	Application shall handle network interruptions gracefully	Reliability Constraint	Reliability	High	Network interruption test
NFR-008	Flutter frontend shall support iOS, Android, and web platforms	Portability Constraint	Portability	High	Cross-platform build and test verification